Estimación de rentabilidad anual para vivienda turística en Madrid

Participantes:

Nazareth Montero
Javier Pascual
Román Diaz
Sara Ruiz

INDICE

1. EDA

- Análisis de target y distribución
- Análisis de variables numéricas
- Correlación con el target
- Análisis variables categóricas
- Eliminación de features preliminar
- Duplicados, missings y outliers
- Feature Engineering

2. Modelado

- Selección de métricas y Cross validation
- Baselines
- Modelado y resultados
- Optimización de hiperparámetros
- Evaluación final y test
- Persistencia del modelo

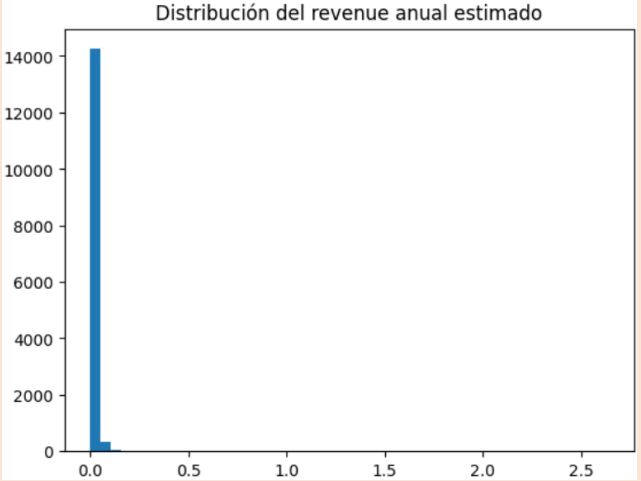
3. Conclusiones

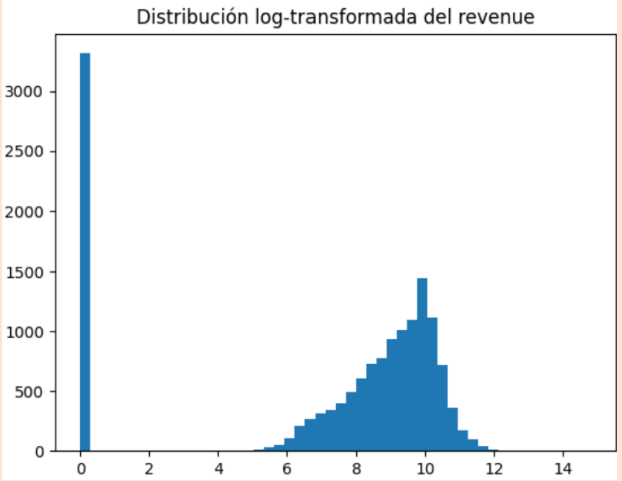
EDA – Análisis de target y su distribución

Elegimos **estimated_revenue_l365d** como variable objetivo, porque resume el ingreso o beneficio anual estimado que puede generar un anuncio. Con esto, buscamos predecir cuánto dinero produce al año cada propiedad del dataset, por lo que utilizaremos un **modelo supervisado de regresión.**

La variable objetivo presenta una distribución fuertemente sesgada a la derecha, con presencia de valores extremos muy elevados. La media es significativamente superior a la mediana, lo que confirma la asimetría. Además, existen valores 0 que deberán analizarse para determinar si corresponden a ausencia de actividad o a posibles errores.

Con el fin de reducir la asimetría, hacer la distribución mas "normal" y reducir el impacto de los outliers vamos a hacer la función logarítmica.





EDA – Análisis de las variables numéricas

Durante el análisis univariante se observa que muchas de las variables numéricas presentan una fuerte asimetría, con distribuciones sesgadas hacia la derecha y presencia de valores extremos. Esto sugiere la necesidad de un tratamiento posterior de outliers y posibles transformaciones.

Se identifican variables potencialmente redundantes, especialmente dentro de los bloques de disponibilidad (**availability_30, availability_60, availability_90, availability_365, availability_eoy**) y de restricciones de estancia (**minimum_nights, maximum_nights y sus derivados como minimum_minimum_nights, maximum_maximum_nights**, etc.). Estas variables miden conceptos muy similares en distintos horizontes temporales o como agregaciones estadísticas, por lo que es esperable que presenten una alta correlación entre sí. Esto puede generar multicolinealidad en modelos lineales, por lo que posteriormente se evaluará la conveniencia de seleccionar únicamente las variables más representativas de cada bloque.

Asimismo, se detectan variables que actúan como identificadores (por ejemplo, **host_id**), las cuales no aportan información predictiva real y deben ser eliminadas del modelado.

Por último, algunas variables como los **review_scores** presentan una variabilidad reducida, concentrándose en valores altos (entre 4 y 5), lo que puede limitar su capacidad explicativa. También se detectan valores extremos en varias variables que deberán analizarse y tratarse en etapas posteriores del preprocesamiento.

EDA – Correlación entre variables numéricas y target

Del análisis bivariante se observa que el bloque de variables con mayor relación con la variable objetivo es el compuesto por la ocupación estimada anual (**estimated_occupancy_l365d**) y las métricas asociadas al volumen de reseñas (**number_of_reviews_ltm, number_of_reviews_ly, number_of_reviews_l30d, reviews_per_month, number_of_reviews**).

Estas variables presentan las correlaciones más elevadas con el revenue log-transformado, lo que sugiere que la intensidad de uso del alojamiento y su nivel de actividad están fuertemente vinculados con los ingresos generados.

Por otro lado, las variables estructurales del alojamiento (como capacidad o características físicas) muestran una correlación moderada, indicando que influyen en el revenue, aunque en menor medida que las variables de actividad.

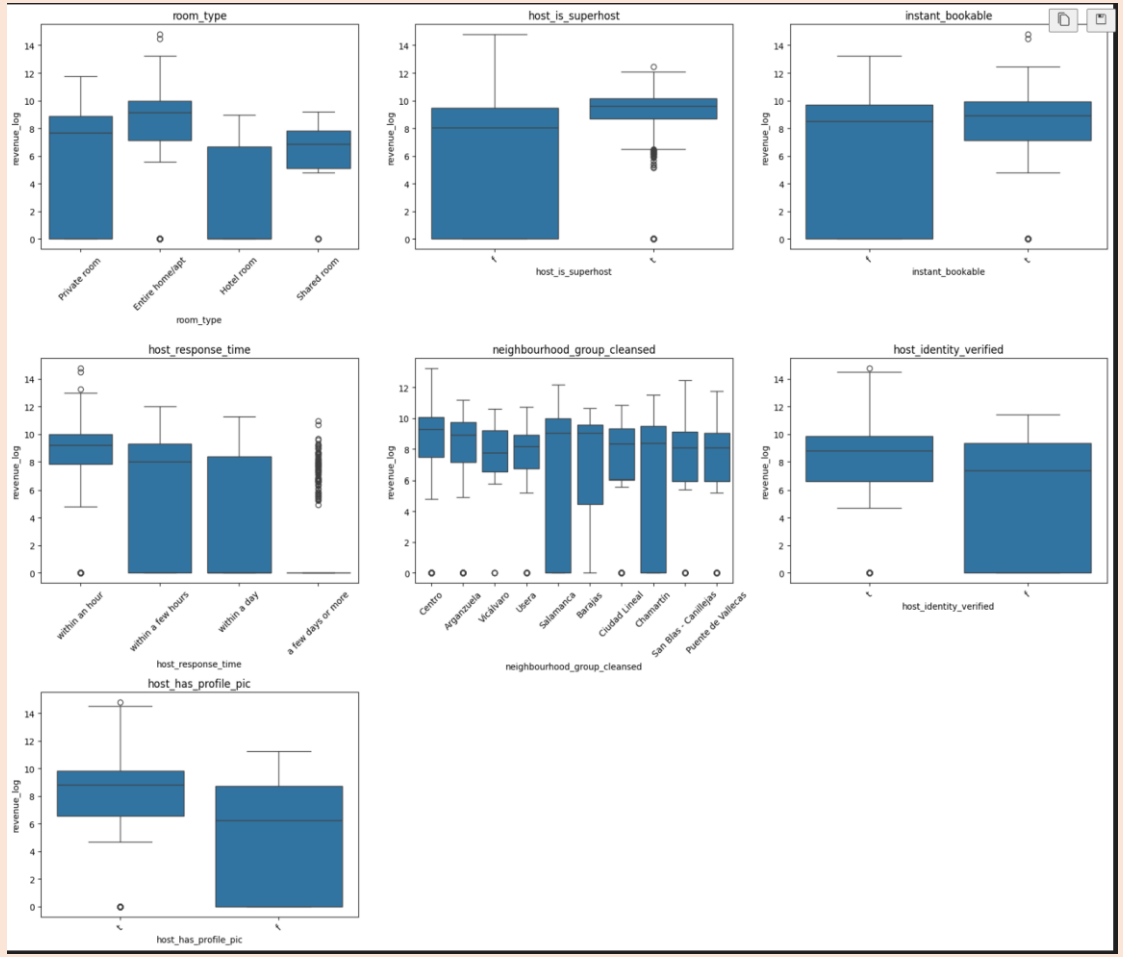
Finalmente, las variables relacionadas con restricciones de estancia (bloque de "**nights**") presentan una relación más débil y potencialmente redundante entre sí, por lo que será necesario evaluar su posible reducción para evitar problemas de multicolinealidad en modelos posteriores.

EDA – Análisis de las variables categóricas

Tras el análisis univariante de las variables categóricas, se ha optado por seleccionar únicamente aquellas con baja o moderada cardinalidad y potencial capacidad explicativa respecto a la variable objetivo.

Se descartan variables de texto libre, fechas técnicas o identificadores, así como aquellas con excesivo número de categorías que podrían generar un alto número de variables dummy y aumentar innecesariamente la complejidad del modelo.

Las variables seleccionadas presentan una estructura interpretable, categorías bien definidas y relevancia conceptual en el contexto del negocio (tipo de alojamiento, características del anfitrión y disponibilidad), lo que las convierte en candidatas adecuadas para el análisis bivariante y su posible inclusión en modelos predictivos posteriores.



EDA – Análisis de las variables categóricas

En el análisis bivariante de variables categóricas se observan diferencias claras en la distribución del revenue según ciertas categorías.

En esta fase se ha realizado un preprocesamiento específico de algunas variables categóricas con el objetivo de prepararlas para el modelado.

En primer lugar, la variable **host_response_time**, que posee un orden natural en sus categorías, ha sido transformada en una variable ordinal numérica. Esta codificación permite preservar la jerarquía implícita en el tiempo de respuesta del anfitrión (desde respuestas más rápidas hasta más lentas), facilitando que los modelos puedan capturar mejor su posible relación con el revenue.

Por otro lado, se ha simplificado la variable **room_type**, agrupando las categorías con muy baja representación ("Shared room" y "Hotel room") bajo una nueva categoría común ("Other"). Esta decisión reduce la dispersión de información y evita que categorías con pocos registros introduzcan ruido en el modelo.

Finalmente, en las variables binarias relacionadas con características del anfitrión (**host_is_superhost**, **host_identity_verified**, **host_has_profile_pic** e **instant_bookable**), se han imputado los valores faltantes con la categoría "Unknown", evitando la pérdida de observaciones y manteniendo la coherencia categórica para su posterior codificación.

EDA – Eliminación de features preliminar

En esta fase se realiza una depuración moderada del conjunto de variables con el objetivo de eliminar información no estructurada y posibles identificadores sin capacidad predictiva, manteniendo, no obstante, aquellas variables potencialmente redundantes que podrían aportar valor en modelos no lineales.

Dado que el análisis contempla tanto modelos lineales como modelos basados en árboles, se opta por una estrategia intermedia: reducir variables claramente irrelevantes sin aplicar una eliminación excesivamente restrictiva, permitiendo posteriormente comparar el comportamiento de distintos algoritmos frente a posibles problemas de multicolinealidad.

```python
# Dejamos unicamente la variable availability_365 del bloque de availability
#  ya que muestran una correlacion muy fuerte.

availability_cols = [
    "availability_30",
    "availability_60",
    "availability_90",
    "availability_eoy"
]

df = df.drop(columns=[c for c in availability_cols if c in df.columns])
```

```python
# Mantenemos las variables number_of_reviews_ltm y reviews_per_month del bloque de reviews.

reviews_drop = [
    "number_of_reviews",
    "number_of_reviews_ly",
    "number_of_reviews_l30d"
]

df = df.drop(columns=[c for c in reviews_drop if c in df.columns])
```

```python
# Mantenemos unicamente minimum_nights y maximum_nights del bloque de nights.

nights_drop = [
    "minimum_minimum_nights",
    "minimum_maximum_nights",
    "maximum_minimum_nights",
    "maximum_maximum_nights",
    "minimum_nights_avg_ntm",
    "maximum_nights_avg_ntm"
]

df = df.drop(columns=[c for c in nights_drop if c in df.columns])
```

EDA – Duplicados, missings y outliers

- Al partir de tres subdatasets que unificamos en el dataset definitivo de partida, la mayoría de duplicados ya los eliminamos previamente localizando coincidencias por id. Por lo que en este punto, teníamos un número reducido de duplicados en nuestro dataframe que eliminamos.

- Continuamos con el tratamiento de nulos o missings, valorando para cada feature la solución más acorde en función del porcentaje de nulos y el tipo de la columna.
  Aplicamos tratamientos que van desde la eliminación de una feature completa al presentar un porcentaje muy elevado de missings y ser de poca relevancia para el objetivo, hasta imputación de medianas, modas, mapeo de datos en función de clasificación de los valores e imputación de 0 u otros valores según la distribución de cada feature.

- Por último, para concluir con esta evaluación preliminar de features, detectamos los posibles outliers.
  En nuestro caso visualizamos los percentiles extremos 1% y 99% para comparar con los valores mínimos y máximos.
  Con esta visualización, igualamos los máximos al valor del percentil 99% de alguna feature que carecía de sentido ese valor máximo.

EDA – Feature engineering

Dado el objetivo de nuestro modelo de Machine Learning, teníamos que tratar las features categóricas en numéricas, de forma que pudiésemos dotarlas de sentido y significado para el entrenamiento del modelo.

Por ello, tratamos individualmente cada columna categórica y fuimos aplicando el encoding más acorde en función de su cardinalidad.

Algunas features con pocos valores únicos era fácilmente transformables en binarias o aplicándoles one-hot-encoding para la generación de estas columnas binarias-numéricas y alguna variable susceptible de mapeo de datos para su conversión en numérica.

En el caso de otras features de tipo texto, extrajimos parte numérica que podía tener sentido para el modelo y también tratamos la parte de texto generando una columna binaria si por ejemplo la fila del dataset tenía una característica o no.

Con todo ello, fuimos transformando las categóricas en numéricas, aportando valor a las features para mejorar el aprendizaje del modelo.
Por último, alguna feature con alta cardinalidad, tuvimos que aplicarle feature encoding para dotarla de sentido para el modelo, por lo que este tratamiento lo aplicamos únicamente a train para no provocar un "*data leakage*" en los datos sobre los que predeciríamos.

Modelado y resultados: Selección de métricas y CV

- Transformamos el target a revenue_log = log1p(revenue) para reducir asimetría y el impacto de outliers en una variable monetaria.
- Elegimos **RMSE** como métrica principal (sobre revenue_log) porque penaliza más los errores grandes y permite comparar modelos de forma consistente.
- Para interpretar resultados en euros, deshacemos la transformación al evaluar: revenue = expm1(revenue_log).
- Establecimos un único protocolo de evaluación con **K-Fold Cross-Validation** (k=5), usando shuffle=True y random_state= 42 para reproducibilidad.
- Regla clave: todos los modelos se evalúan con la **misma CV y la misma métrica**, evitando comparaciones sesgadas.

Modelado y resultados (Código/Baselines)

```python
pipelines = {"Ridge": Pipeline([("imputer", SimpleImputer(strategy="median")),
                                ("scaler", StandardScaler()),
                                ("model", Ridge(alpha= 10.0))]),

                                "RandomForest": Pipeline([("imputer", SimpleImputer(strategy="median")),
                                                          ("model", RandomForestRegressor(n_estimators= 400,
                                                                                          random_state= 42,
                                                                                          n_jobs= -1))]),

                                "XGBoost": Pipeline([("imputer", SimpleImputer(strategy="median")),
                                                     ("model", XGBRegressor(n_estimators= 600,
                                                                            learning_rate= 0.05,
                                                                            max_depth= 6,
                                                                            subsample= 0.9,
                                                                            colsample_bytree= 0.9,
                                                                            reg_lambda= 1.0,
                                                                            random_state= 42,
                                                                            n_jobs= -1))])
                                                                                          }

results = {}

for name, pipe in pipelines.items():
    rmse = -cross_val_score(pipe, X_train, y_train, cv= cv, scoring= "neg_root_mean_squared_error").mean()
    results[name] = rmse

    print(f"CV RMSE (log) {name}: {rmse:.3f}")

best_name = min(results, key= results.get)
best_pipe = pipelines[best_name]
                                (variable) best_name: Any
print(f"\nMejor por CV (log RMSE): {best_name} -> {results[best_name]:.3f}")
```

```
CV RMSE (log) Ridge: 1.273
CV RMSE (log) RandomForest: 0.068
CV RMSE (log) XGBoost: 0.054
```

```python
y_test_eur = np.expm1(y_test)
print("Median revenue (test):", np.median(y_test_eur))
print("Mean revenue (test):  ", np.mean(y_test_eur))
```

```
Median revenue (test): 6840.000000000003
Mean revenue (test):   13382.612418969635
```

```python
y_test_eur = np.expm1(y_test)
for p in [50, 75, 90, 95, 99]:
    print(p, np.percentile(y_test_eur, p))
print("max", y_test_eur.max())
```

```
50 6840.000000000003
75 19403.999999999985
90 31109.99999999997
95 42330.00000000003
99 84353.99999999983
max 1966151.9999999984
```

```python
XGBoost_model = best_pipe
XGBoost_model.fit(X_train, y_train)

y_pred_log = XGBoost_model.predict(X_test)

rmse_test_log = root_mean_squared_error(y_test, y_pred_log)
print(f"Test RMSE (log) XGBoost: {rmse_test_log:.3f}")

rmse_test_eur = root_mean_squared_error(np.expm1(y_test), np.expm1(y_pred_log))
print(f"Test RMSE (€)   XGBoost: {rmse_test_eur:.2f}")
```

```
Test RMSE (log) XGBoost: 0.072
Test RMSE (€)   XGBoost: 29473.08
```

Modelado y resultados

- Pipelines por modelo: mismo preprocesado (imputación; escalado solo en lineales) para evitar leakage y comparar de forma justa.
- Baseline y comparativa: probamos Ridge (baseline), Random Forest y XGBoost con K-Fold CV (k=5) y RMSE sobre revenue_log.
- Selección del modelo: el menor RMSE en CV fue XGBoost, por lo que se eligió para entrenamiento final y posterior optimización.
- Chequeo del target en euros: calculamos media, mediana y percentiles (tras expm1) para entender la magnitud real del revenue y poder interpretar el error.
- Interpretación del RMSE (€): el revenue tiene cola muy pesada (p99 ≈ 84k€ y máximo ≈ 2M€). Como RMSE penaliza errores al cuadrado, estos casos extremos pueden inflar el RMSE en euros aunque el modelo funcione bien para la mayoría.

Modelado y resultados: Optimización de hiperparámetros

Tras construir varios modelos base (baseline), el siguiente paso consiste en mejorar su rendimiento ajustando sus hiperparámetros. La optimización busca encontrar la combinación que minimiza el error y mejora la capacidad de generalización.

En este paso, se han optimizado el modelo lineal, Random Forest y XGBoost con el objetivo de maximizar el rendimiento de los modelos desarrollados durante la fase de modelado, garantizando al mismo tiempo su capacidad de generalización.

Para ello, se ha optado por el uso de RandomizedSearchCV frente a otras alternativas como Grid Search, debido a su mayor eficiencia computacional. Este método permite explorar el espacio de hiperparámetros de forma aleatoria, evaluando un número limitado de combinaciones pero cubriendo un rango amplio de valores.

La métrica seleccionada para la optimización ha sido el error cuadrático medio (RMSE), dado que el problema abordado es de regresión sobre variables continuas relacionadas con precios.

Finalmente, la validación cruzada con 5 particiones ha permitido obtener una estimación robusta del rendimiento del modelo, reduciendo la dependencia de una única partición de los datos y mejorando la fiabilidad de los resultados obtenidos.

Modelado y resultados: Evaluación final en test

La evaluación final se ha realizado sobre el conjunto de test, no utilizado durante el entrenamiento ni en la optimización de hiperparámetros. Esto garantiza una estimación realista del rendimiento del modelo en datos no vistos.

```python
from sklearn.metrics import root_mean_squared_error, r2_score

# Predicción sobre test
y_pred = best_pipe.predict(X_test)

# Métricas
rmse_test = root_mean_squared_error(y_test, y_pred)
r2_test = r2_score(y_test, y_pred)

print(f"RMSE (test): {rmse_test:.2f}")
print(f"R2 (test): {r2_test:.4f}")
```

```python
# RMSE en euros (escala original)
rmse_test_eur = root_mean_squared_error(np.expm1(y_test), np.expm1(y_pred_log))
print(f"RMSE (test, €): {rmse_test_eur:.2f}")
```

Modelado y resultados: Persistencia del modelo

El modelo final optimizado ha sido persistido mediante la utilización de joblib lo que permite su reutilización en entornos productivos y análisis posteriores sin necesidad de reentrenamiento. Esto es fundamental para garantizar la reproducibilidad del proyecto.

```
import joblib

joblib.dump(best_pipe, "modelo_optimizado.pkl")
```

```
['modelo_optimizado.pkl']
```

```
modelo_cargado = joblib.load("modelo_optimizado.pkl")

predicciones_modelo = modelo_cargado.predict(X_test)
```

5. Conclusiones

El proyecto ha seguido un flujo completo de análisis de datos y modelado predictivo, comenzando con la exploración inicial de los datos (EDA), continuando con el preprocesado y la ingeniería de variables, y finalizando con el entrenamiento, optimización y evaluación de modelos de Machine Learning.

Este enfoque permitió:

- Comprender la estructura de los datos
- Identificar patrones relevantes
- Preparar adecuadamente las variables para el modelado
- Construir modelos capaces de generar predicciones fiables.

El preprocesado de los datos fue una fase clave para mejorar la calidad del modelo. Durante esta etapa se realizaron diversas tareas como: tratamiento de valores nulos, transformación de variables categóricas, conversión de variables numéricas, selección de variables relevantes…

Estas transformaciones permitieron construir un conjunto de datos más consistente y adecuado para el entrenamiento de modelos de aprendizaje automático.

5. Conclusiones

Durante el proceso de modelado se evaluaron diferentes algoritmos de regresión con el objetivo de identificar cuál ofrecía el mejor rendimiento predictivo. Para ello se utilizó una **pipeline de procesamiento.**

Posteriormente con la **optimización de hiperparámetros** encontramos la configuración del modelo que ofrecía el mejor equilibrio entre capacidad de ajuste y generalización.

El modelo optimizado se evaluó utilizando un conjunto de datos de test independiente para medir su capacidad de generalización.

Los resultados obtenidos muestran que el modelo es capaz de capturar una parte significativa de la variabilidad de los precios, ofreciendo predicciones razonablemente precisas dentro del contexto del problema analizado.

A pesar de los resultados obtenidos, el modelo presenta algunas limitaciones que deben tenerse en cuenta como que los datos utilizados corresponden a una fotografía temporal concreta del mercado, lo que puede limitar la capacidad de generalización a otros periodos. Otra limitación es que el mercado inmobiliario y turístico está influido por factores externos (regulación, turismo, economía local) que pueden cambiar con el tiempo.